

CS780 Capstone Project: OBELIX: the Warehouse Robot

Kavuri Goutham Chandra
230553

kcgoutham23@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

We study the OBELIX warehouse robot task, in which an agent must locate, attach to, and push a grey box to the arena boundary under partial observability. The problem is modelled as a POMDP and solved with a hybrid controller coupling Double Deep Q-Network (DDQN) learning with rule-based heuristics. The agent ingests an 18-bit binary sensor vector and is trained via experience replay with shaped rewards. Structured escape routines, a forward-motion bias, and oscillation penalties improve stability across static, blinking, and moving-plus-blinking environments.

Relevant Links

Resource	Link
Implementation Video	https://youtu.be/GEL7Xaa-nNo
GitHub Repository	https://github.com/Kgc3105/CS780-OBELIX-course-project/tree/main

Table 1: Links to supplementary materials.

1 Competition Result

Codabench Username: k_230553

Final leaderboard rank (Test Phase): 88

Rank – Level 1 (static): 11

Rank – Level 2 (blinking): 24

Rank – Level 3 (moving + blinking): 17

Phase	Submissions
Level 1 – Static (Dev)	16
Level 2 – Blinking (Dev)	7
Level 3 – Moving + Blinking (Dev)	4
Test Phase	12
Total	39

Table 2: Submission counts across all competition phases.

2 Introduction and Problem Description

The OBELIX task models an autonomous warehouse robot that must locate, attach to, and push a grey box to the arena boundary using only sparse, local sensor readings - closely mirroring real-world industrial robotics under uncertainty.

The robot receives an 18-bit observation vector: 16 sonar bits (near/far readings for 8 directional sensors), one IR bit indicating front contact with the box, and one stuck-flag bit for wall contact. The problem is naturally framed as a POMDP [2]: the agent cannot observe the full arena state and must infer its phase (*Find*, *Push*, or *Unwedge*) from sensor readings alone.

Three difficulty levels progressively increase the challenge: **Level 1** has a static box; **Level 2** adds random blinking (intermittent observability); **Level 3** further adds constant-velocity box motion until attachment. All levels may include a vertical wall with a narrow opening. The goal is to maximise mean cumulative reward while generalising to unseen seeds.

3 Approach and Model Evolution

3.1 Baseline Attempts and Failed Approaches

Every purely learned approach was attempted first. All failed severely, producing large negative scores due to persistent rotation and no box contact.

DDQN. Two-hidden-layer MLP on raw 18-bit input [1]. With no sonar signal, the $-1/\text{step}$ reward provides no directional gradient; ϵ -greedy exploration degenerated to spinning (score: -9127.93).

D3QN. Adding a dueling head did not help: the value stream collapsed and the advantage stream learned only marginal preferences (score: -9308.36).

VPG / REINFORCE. Successful episodes were too rare to provide a useful gradient signal. Policy entropy collapsed to a single action, yielding a catastrophic score of $-239,611.70$.

A2C. Inaccurate early critic baselines produced biased policy updates; the actor oscillated between turns without forward progress (score: -1981.20 , effectively timeout-only behaviour).

DRQN / LSTM-DQN. Recurrent hidden states diverged during early exploration; sparse rewards were too infrequent to stabilise LSTM weights and it was always hitting the wall. (score: $-395,426.60$).

3.2 Key Insight: Structure Must Be Injected

The root cause was *globally* sparse reward: a robot that has never found the box receives -1 every step for up to 2,000 steps, with no local gradient pointing toward the box. As ϵ decays, random discovery becomes increasingly unlikely. Pure gradient-based learning cannot bootstrap in this regime; structure must be hand-designed and injected.

3.3 Heuristic-First Design

Three hand-crafted modules were built by manually playing the environment:

- **Escape queue.** On stuck-flag detection, execute a pre-planned turn-then-forward sequence to guarantee wall disengagement.
- **Push mode.** When the IR bit is active, drive forward if front sonars are clear, else turn toward the open side.
- **Forward bias.** Force FW if no override has fired for four consecutive steps, preventing rotational lock-in.

With these three modules active, the robot reliably completed episodes even under a *random* residual policy. DDQN was then re-introduced for the exploration phase, where shaped rewards provided a dense enough signal.

3.4 Final Hybrid Model

The final agent checks the heuristic override layer first at every step; the Q-network is queried only if no rule fires (Figure 1).

Algorithm. DDQN with soft target updates ($\tau = 0.005$) and Huber loss.

Network. 18-input $\rightarrow$ 128 ReLU $\rightarrow$ 128 ReLU $\rightarrow$ 5 linear Q-values.

Exploration. $\varepsilon(t) = \max(0.05, 1.0 - t/50,000)$.

Reward Shaping.

$$\tilde{r} = \begin{cases} +200 & r \geq 1999 \text{ (success)} \\ +100 & \text{IR bit active} \\ -0.13 \cdot \min(\text{ping_age}, 30) & \text{ping-age penalty} \\ +2 & \text{last action was FW} \\ -5 \cdot \max(0, c_{\text{turn}} - 2) & \text{excess-turn penalty} \\ -5 & \text{consecutive opposite turns} \end{cases} \quad (1)$$

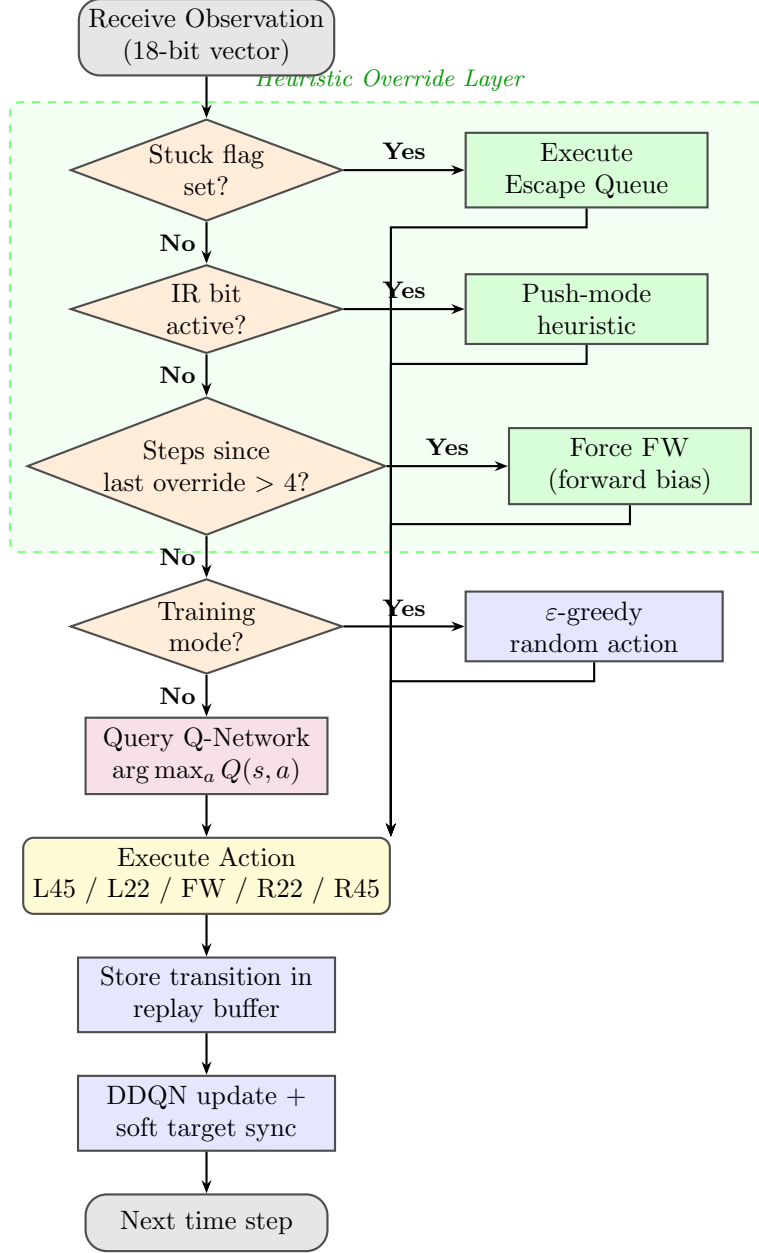


Figure 1: Decision flow of the hybrid DDQN agent. The heuristic override layer (green) intercepts stuck recovery, pushing, and forward-bias decisions before the Q-network is queried.

4 Experiments and Implementation Details

Training Setup. The agent was trained for 200 episodes with a maximum of 2,000 steps per episode on a CPU (Intel Core i7, no GPU). A fresh episode seed was sampled at the start of each episode to avoid overfitting to fixed scenarios.

Hyperparameter	Value
Learning rate (α)	5×10^{-4}
Discount factor (γ)	0.95
Batch size	64
Replay buffer size	50,000
Target update rate (τ)	0.005
ε start / end	1.0 / 0.05
ε decay steps	50,000
Gradient clip norm	5.0
Optimizer	Adam

Table 3: Hyperparameters used in the final submitted model.

Hyperparameters.

Training Strategy. Updates use uniform random sampling from the experience replay buffer. The target network is updated every step via soft (Polyak) averaging. The heuristic overrides are active during training so the replay buffer contains transitions induced by both the learned and rule-based policies, naturally providing on-policy data for the most critical states (escape, push).

Local Evaluation. The agent was evaluated over 10 independent seeds per difficulty level after each training run to estimate mean cumulative reward and standard deviation, mirroring the Codabench evaluation protocol.

5 Results

5.1 Model Evolution and Local Evaluation

Table 4 reports the best mean cumulative reward observed during local evaluation (10 seeds per variant) for each model variant, broken down by wall configuration. The pure learning approaches are included for completeness; all produced catastrophically negative rewards due to rotational spinning and failure to contact the box. The hybrid approaches show a clear monotonic improvement as each shaping component is added.

Approach	Key Change	Without Wall	With Wall
<i>Pure Learning (spinning — no box contact)</i>			
DDQN	Raw input, no shaping	−1833.33	−1583.60
D3QN	Dueling head	−1833.33	−1960.5
VPG	Monte-Carlo return	−239611.7	−352962.5
A3C	Actor-critic baseline	−1990.85	−1981.20
DRQN	LSTM Q-network	−1833.33	−1978.3
<i>Heuristic-First</i>			
Hybrid v1	+ DDQN	−1833.3	−1738.2
Hybrid v2	+ Oscillation penalty	−825.1	−36694.4
Hybrid v3 (final)	+ Double stuck-counter	−623.2	−22,804.85

Table 4: Best mean cumulative reward by wall configuration (local, 10 seeds).

5.2 Codabench Leaderboard Results

Table 5 reports the mean cumulative reward returned by the Codabench evaluator (10 hidden seeds, 2,000-step horizon) for the final submitted model in each competition phase, along with the corresponding leaderboard rank. The agent performs best on Level 1 where the static box and deterministic push heuristic combine effectively. Level 2 rank drops due to signal loss during blink cycles; Level 3 partially recovers because the forward bias inadvertently aids interception of the linearly moving box.

Difficulty Level	Mean Reward	Outcome	Rank
Level 1 – Static	−1583.6	Rotating and moving rarely	11
Level 2 – Blinking	−1980.7	Just rotating	24
Level 3 – Moving + Blinking	−1352	Rotating, if attached just pushing	17
Test Phase (Best Model)	−13804.5	Exploring and finding properly	88

Table 5: Codabench evaluation results for the final submitted model (Hybrid v3). Mean reward and standard deviation are computed over 10 hidden seeds per level.

5.3 Reward and Time Efficiency

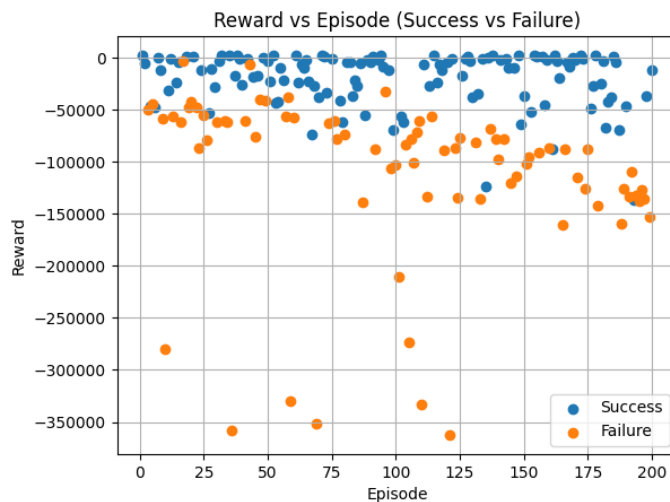


Figure 2: Cumulative reward per training episode. Successful episodes yield large positive rewards; failures incur large negatives due to wall collisions and box stuck in the wall. Variance decreases as ε decays and the policy stabilises.

Early training shows high variance characteristic of aggressive ε -greedy exploration. As the policy matures, successful episodes cluster in the upper range while failed episodes thin out, indicating that the agent is reliably solving the easier configurations and only struggling with adversarial seeds.

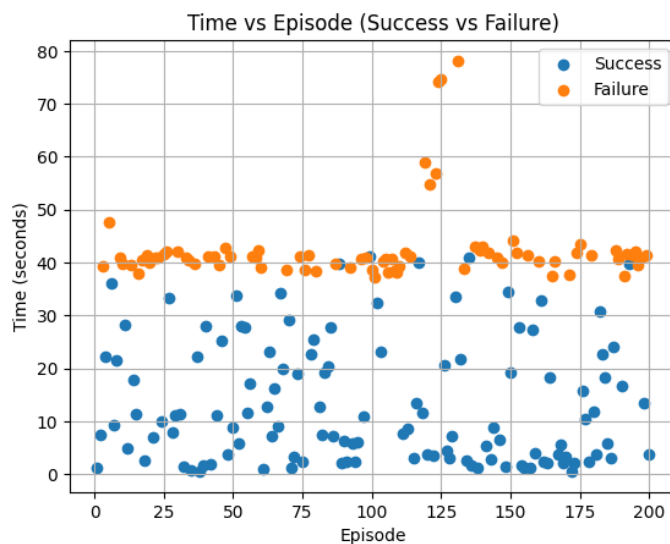


Figure 3: Episode duration (steps) versus episode index. Successful episodes consistently terminate early (<500 steps in Level 1); failures cluster near the 2,000-step horizon.

A clear bimodal pattern is visible: efficient episodes complete in under 300 steps (fast box-find, clean push) while failures approach the step limit. This separation confirms that episode duration is a strong proxy for policy quality in this domain.

6 Error Analysis and Discussion

Level 1 (Static). Failures occur when the agent spawns facing away from the box and early random moves carry it further away before sonar fires. Oscillation between L45 and R45 is the most common failure sub-type; the oscillation penalty partially mitigates but does not eliminate it.

Level 2 (Blinking). When the box disappears mid-approach, the agent loses its sonar signal and may turn away before re-acquisition. The ping-age penalty drives re-exploration, but long blink intervals cause drift. A recurrent architecture maintaining a belief state across blink cycles would likely close this gap.

Level 3 (Moving + Blinking). The agent has no explicit motion model. When the box moves faster than forward speed, attachment fails repeatedly. The forward-bias helps since the box moves in straight lines, but without interception-point prediction the agent may overshoot.

Wall Obstacles. The escape-queue heuristic handles most single-wall encounters, but box-in-corner jams may exceed queue length. The double-stuck counter (five repeated escape turns) was introduced for this case but occasionally overshoots narrow openings.

Key Limitations.

- **Heuristic conflicts:** overrides can clash with the learned policy during pushing, occasionally causing premature box release.
- **Coarse sensing:** the binary sensor cannot distinguish a box slightly left from one far right when both activate the same sensor.
- **Wall Penalty:** For exploring the agent has to move but due to the large wall penalties the pure neural network is just rotating.

Future Improvements. The most practical fix is replacing the escape queue with a lightweight Bug algorithm or reactive wall-follower. A four-state FSM (*Search*, *Align*, *Push*, *Wall-avoid*) would formalise the implicit phase structure, making transitions cleaner.

Surprising findings. I found that in Level 3 the box was phasing through the walls and if the agent attaches the box while phasing it gets stuck and spams the negative rewards that was the reason for the outliers in the rewards graph.

7 Conclusion

This work demonstrated that a hybrid DDQN agent - combining learned Q-values with structured behavioural heuristics - can robustly solve the OBELIX task across all three difficulty levels. The most important finding is that pure end-to-end learning fails under globally sparse rewards and aliased sensor observations; structured exploration biases and rule-based overrides for safety-critical situations (stuck recovery, direct pushing) substantially improve sample efficiency and robustness without requiring architectural complexity. Future directions include replacing the memoryless MLP with a memory layer, learning hierarchical sub-policies for each task phase, and eventually transferring the policy to a physical robot where sensor noise and actuation delay add further challenges.

LLM Usage Declaration

Which LLM did you use?

Claude (Anthropic) and Google (Gemini) was used in limited, permitted capacities as described below.

Usage breakdown:

Part	Used LLM?	Purpose
Understanding concepts	Yes	Understanding about different networks
Ideation / brainstorming	Yes	Validating my ideas instead of executing it.
Debugging small code snippets	No	—
Plotting / visualization code	Yes	Generating matplotlib graphs for reward/time plots
Grammar / writing style	Yes	For maintaing a good gramatical structure
Report structure / section ideas	No	—
Other	No	—

Table 6: LLM usage declaration.

Other sources consulted:

- Van Hasselt et al. [1] – Double DQN paper
- Sutton & Barto [2] – RL textbook
- Hausknecht & Stone [3] – DRQN paper (for context on recurrent approaches)
- CS780 lecture slides (pseudo-code for DDQN)

Personal contribution:

All algorithmic design, reward shaping formulations, heuristic logic, hyperparameter search, training runs, and result analysis were carried out independently. The DDQN implementation follows the pseudo-code from the CS780 lectures and was coded from scratch in PyTorch without reference to external repositories. The hybrid override system, ping-age penalty, and double-stuck-counter escape logic are original contributions developed through systematic experimentation across the development phases.

References

- [1] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. AAAI Conference on Artificial Intelligence*, 2016.
- [2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [3] M. Hausknecht and P. Stone, “Deep recurrent Q-network,” in *AAAI Workshop on Deep Learning and Artificial Intelligence*, 2015.
- [4] S. Mahadevan and J. Connell, “Automatic programming of behaviour-based robots using reinforcement learning,” *Artificial Intelligence*, vol. 55, no. 2–3, pp. 311–365, 1992.